```
  def total_value_in_stock
    # later we'll see easier ways to sum a collection
    sum = 0.0
    @books_in_stock.each { |book| sum += book.price_in_cents }
    sum / 100.0
  end

  def number_of_each_isbn
    # ...
  end
end
```

In this file, we use the require method to load in the Ruby CSV library from the Ruby standard library and we use require_relative to load in the book_in_stock file we wrote. We use require_relative for this because the location of the file we're loading is easiest to define relative to the file we're loading it from—they're both in the same directory.

We're using price_in_cents to compute the total value.

And finally, here's our main program, in the file stock_stats.rb:

tut_classes/stock_stats/stock_stats.rb

```
require_relative "csv_reader"

reader = CsvReader.new

ARGV.each do |csv_file_name|
  $stderr.puts "Processing #{csv_file_name}"
  reader.read_in_csv_data(csv_file_name)
end

puts "Total value = #{reader.total_value_in_stock}"
```

Again, this file uses require_relative to bring in the library it needs (in this case, the csv_reader.rb file). It uses the ARGV variable to access the program's command-line arguments, loading CSV data for each file specified on the command line.

We can run this program using the simple CSV data file as we demonstrated on page 33:

```
$ ruby stock_stats.rb data.csv
Processing data.csv
Total value = 122.07
```

Do we need three source files for this? Not necessarily. But as your programs grow (and almost all programs grow over time), you'll find that large files start to get cumbersome. You'll also find it harder to write automated tests against the code if it is in a monolithic chunk. Finally, you won't be able to reuse classes if they're all bundled into the final program. As a result, it's fairly common to only have one Ruby class per individual file.

Let's get back to our discussion of classes.

Specifying Access Control

When designing a class interface, it's important to consider just how much of your class you'll be exposing to the outside world. Allow too much access into your class, and you risk increasing the amount that different classes depend on each other's internal implementation, which is called *coupling*. Users of your class will be tempted to rely on details of your class's